NCAA Champion Checklist

DATA 5600 — Milestone 15: Initial Logistic Regression

Research question and objective

Research question: Which pre-tournament team statistics are associated with winning the NCAA men's basketball national championship, and how could these relationships help us evaluate the MacFax Champion Checklist?

Milestone 15 prompt: *Using the data for your project, run an initial model and interpret the parameter estimates. What story is starting to take shape?*

We use logistic regression because the response, `champion`, is binary: it equals **1** for the national champion and **0** for everyone else. Our first goal is to estimate how adjusted efficiency margin (AdjEM) relates to championship odds. We then check whether regular-season win percentage adds information and compare an alternative specification that separates offensive and defensive efficiency.

The regression is a **statistical benchmark**, not the existing 15-criterion MacFax checklist. A useful next stage of the project will be to compare the checklist against these regression models on later seasons.

Data and modeling approach

Our group acquired historical NCAA basketball data from the MacFax PostgreSQL database and exported the relevant records to `data/processed/champion_checklist.csv`. Each row represents one team in one season. The shared export covers **2005–2026, excluding 2020**, when the NCAA tournament was canceled. We focus on teams with a recorded NCAA tournament seed and use the ratings marked as *pre-tournament*.

The original export contains 7,311 team-season records and 45 variables. Our initial analysis uses 1,343 recorded tournament-team seasons, including 21 national champions. The exported field is not fully complete in every year, and we have not independently audited how all historical

pre-tournament ratings were generated. These are important limitations when interpreting results as evidence about future tournaments.

Load and prepare the dataset

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from IPython.display import Markdown, display
from scipy.stats import chi2
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Use the CSV already shared with the group; no database connection is needed.
candidates = [
    Path("data/processed/champion_checklist.csv"),
    Path("../data/processed/champion_checklist.csv"),
]
data_path = next((p for p in candidates if p.is_file()), None)
if data_path is None:
    raise FileNotFoundError(
        "Could not find data/processed/champion_checklist.csv. "
        "Place this QMD in the project root or one folder below it."
    )

raw = pd.read_csv(data_path)
required = {
    "season", "team", "team_id", "tournament_seed", "champion",
    "adj_em", "win_pct", "adj_o", "adj_d"
}
missing_columns = required - set(raw.columns)
if missing_columns:
    raise ValueError(f"Missing expected CSV columns: {sorted(missing_columns)}")

# Filter to actual tournament participants; preserve the original raw data.
data = raw.loc[raw["tournament_seed"].notna()].copy()
for col in ["season", "champion", "adj_em", "win_pct", "adj_o", "adj_d"]:
    data[col] = pd.to_numeric(data[col], errors="coerce")
```

```python
before_missing = len(data)
data = data.dropna(
    subset=["season", "champion", "adj_em", "win_pct", "adj_o", "adj_d"]
).copy()
data["season"] = data["season"].astype(int)
data["champion"] = data["champion"].astype(int)

# Interpret one unit as a meaningful difference in the basketball statistics.
# Fixed reference points keep the intercept interpretable and avoid leakage
# from using a held-out year's mean during chronological validation.
data["adj_em_5"] = (data["adj_em"] - 20) / 5
data["win_pct_10pp"] = (data["win_pct"] - 0.75) / 0.10
data["adj_o_5"] = (data["adj_o"] - 110) / 5
data["adj_d_5"] = (data["adj_d"] - 100) / 5

assert data["champion"].isin([0, 1]).all(), "Outcome must be binary."
assert not data.duplicated(["season", "team_id"]).any(), "Duplicate team-season."
assert data.groupby("season")["champion"].sum().eq(1).all(), (
    "Expected exactly one champion in each included season."
)

print(f"CSV: {data_path}")
print(f"Original CSV observations: {len(raw):,}")
print(f"Tournament-team observations used: {len(data):,}")
print(f"Observations dropped for missing model inputs: {before_missing - len(data)}")
print(f"Seasons: {data['season'].nunique()}")
print(f"Champions: {int(data['champion'].sum())}")
print(f"Non-champions: {int((data['champion'] == 0).sum())}")
print(f"Champion rate: {100 * data['champion'].mean():.2f}%")
```

```
CSV: ../data/processed/champion_checklist.csv
Original CSV observations: 7,311
Tournament-team observations used: 1,343
Observations dropped for missing model inputs: 0
Seasons: 21
Champions: 21
Non-champions: 1322
Champion rate: 1.56%
```

Because only one team wins each season, we have far fewer positive outcomes than non-champions. We therefore avoid fitting all 15 checklist criteria (or all 45 exported columns) in one

ordinary logistic regression. We also avoid including AdjEM, AdjO, and AdjD **simultaneously** because AdjEM is computed from offense minus defense, creating an identifiability problem.

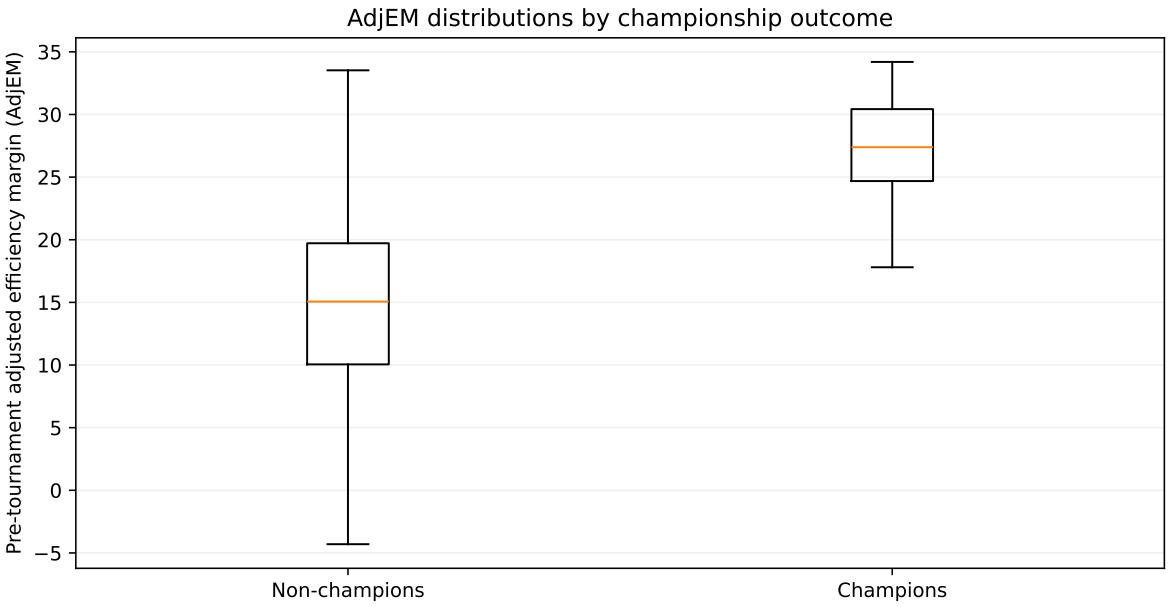
Initial descriptive comparison

Before fitting the model, we compare the average statistics of champions and non-champions. These descriptions do not control for other variables.

```
summary = data.groupby("champion").agg(
    teams=("team", "count"),
    mean_adj_em=("adj_em", "mean"),
    mean_adj_o=("adj_o", "mean"),
    mean_adj_d=("adj_d", "mean"),
    mean_win_pct=("win_pct", "mean"),
).rename(index={0: "Non-champion", 1: "Champion"})
display(summary.round(3))

fig, ax = plt.subplots(figsize=(8.5, 4.5))
ax.boxplot(
    [data.loc[data.champion == 0, "adj_em"],
     data.loc[data.champion == 1, "adj_em"]],
    tick_labels=["Non-champions", "Champions"],
    showfliers=False,
)
ax.set(ylabel="Pre-tournament adjusted efficiency margin (AdjEM)",
       title="AdjEM distributions by championship outcome")
ax.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.show()
```

champion	teams	mean_adj_em	mean_adj_o	mean_adj_d	mean_win_pct
Non-champion	1322	14.277	110.714	96.436	0.725
Champion	21	27.549	119.395	91.846	0.850



Initial logistic regression models

We estimate three deliberately small models. The first is our baseline; the other two ask whether more specific descriptions of team strength improve our understanding.

Model	Predictors	Question
1 — Efficiency baseline	AdjEM	Are stronger teams, as measured by net efficiency, more likely to win?
2 — Efficiency + record	AdjEM and win percentage	Does winning more regular-season games add information beyond efficiency?
3 — Offense + defense	AdjO and AdjD	Do offense and defense show separate relationships with championship outcomes?

The general logistic model is

Each coefficient represents a change in **log-odds**, holding the other predictors in that model fixed. Exponentiating a coefficient gives an **odds ratio**. An odds ratio is *not* a percentage-point change in championship probability.

Model 1 — Adjusted efficiency margin

Our initial model is

We scale AdjEM so that a one-unit increase in the model represents **five additional efficiency-margin points**. The intercept describes a team with AdjEM = 20, not an average team in every other respect.

```
family = sm.families.Binomial()
m1 = smf.glm("champion ~ adj_em_5", data=data, family=family).fit()
print(m1.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:               champion   No. Observations:                 1343
Model:                            GLM   Df Residuals:                     1341
Model Family:                Binomial   Df Model:                            1
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -
72.941
Date:                Sat, 31 Oct 2026   Deviance:                       145.88
Time:                        11:05:54   Pearson chi2:                     590.
No. Iterations:                     9   Pseudo R-squ. (CS):            0.05109
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -4.6430      0.376    -12.343      0.000      -5.380      -
3.906
adj_em_5       1.5098      0.221      6.834      0.000       1.077       1.943
==============================================================================
```

Model 2 — AdjEM and win percentage

The second model adds win percentage to see whether a team's win-loss record contains information beyond its net efficiency. A one-unit increase in `win_pct_10pp` means **10 percentage points** higher win percentage (for example, 70% to 80%).

```
m2 = smf.glm(
    "champion ~ adj_em_5 + win_pct_10pp", data=data, family=family
).fit()
print(m2.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                champion   No. Observations:                 1343
Model:                            GLM   Df Residuals:                     1340
Model Family:                Binomial   Df Model:                            2
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -
72.924
Date:                Sat, 31 Oct 2026   Deviance:                       145.85
Time:                        11:05:54   Pearson chi2:                     588.
No. Iterations:                     9   Pseudo R-squ. (CS):            0.05111
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      -4.6502      0.379    -12.280      0.000      -5.392      -
3.908
adj_em_5        1.4666      0.319      4.603      0.000       0.842       2.091
win_pct_10pp    0.0736      0.396      0.186      0.853      -0.702       0.849
==============================================================================
```

Model 3 — Offense and defense separately

AdjEM summarizes overall efficiency, but two teams with the same AdjEM can have different offensive and defensive profiles. Our third model replaces AdjEM with AdjO and AdjD; it does **not** include all three together.

For AdjO, a higher value means a stronger offense. For AdjD, a **lower** value means a stronger defense. Both coefficients are measured per five rating points.

```
m3 = smf.glm(
    "champion ~ adj_o_5 + adj_d_5", data=data, family=family
).fit()
print(m3.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                champion   No. Observations:                 1343
Model:                            GLM   Df Residuals:                     1340
Model Family:                Binomial   Df Model:                            2
Link Function:                  Logit   Scale:                          1.0000
```

```
Method:                           IRLS   Log-Likelihood:                -
72.886
Date:                Sat, 31 Oct 2026   Deviance:                       145.77
Time:                        11:05:54   Pearson chi2:                     600.
No. Iterations:                     9   Pseudo R-squ. (CS):            0.05117
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -7.6104      0.782     -9.728      0.000      -9.144      -
6.077
adj_o_5        1.5484      0.251      6.179      0.000       1.057       2.039
adj_d_5       -1.4358      0.312     -4.600      0.000      -2.047      -
0.824
==============================================================================
```

Interpret the parameter estimates

Coefficients, odds ratios, and uncertainty

We report coefficient estimates, ordinary model-based 95% confidence intervals, and odds ratios. Because teams share a tournament and programs recur across years, the usual independent-observation standard errors and p-values should be treated as **exploratory**, not definitive.

```python
def coefficient_table(result):
    ci = result.conf_int()
    return pd.DataFrame({
        "Coefficient (log-odds)": result.params,
        "Odds ratio": np.exp(result.params),
        "OR 95% CI lower (naive)": np.exp(ci.iloc[:, 0]),
        "OR 95% CI upper (naive)": np.exp(ci.iloc[:, 1]),
        "P-value (exploratory)": result.pvalues,
    }).round(4)

for label, fit in [
    ("Model 1: AdjEM", m1),
    ("Model 2: AdjEM + win percentage", m2),
    ("Model 3: AdjO + AdjD", m3),
]:
    print(f"\n{label}")
    display(coefficient_table(fit))
```

```
print(f"Model 1, +5 AdjEM: odds multiplied by {np.exp(m1.params['adj_em_5']):.2f}")
print(f"Model 2, +5 AdjEM (holding win% fixed): "
      f"odds multiplied by {np.exp(m2.params['adj_em_5']):.2f}")
print(f"Model 2, +10 percentage points win percentage (holding AdjEM fixed): "
      f"odds multiplied by {np.exp(m2.params['win_pct_10pp']):.2f}")
print(f"Model 3, +5 AdjO (holding AdjD fixed): "
      f"odds multiplied by {np.exp(m3.params['adj_o_5']):.2f}")
print(f"Model 3, +5 AdjD (holding AdjO fixed): "
      f"odds multiplied by {np.exp(m3.params['adj_d_5']):.2f}")
```

```
Model 1: AdjEM
```

	Coefficient (log-odds)	Odds ratio	OR 95% CI lower (naive)	OR 95% CI upper (naive)	P-va
Intercept	-4.6430	0.0096	0.0046	0.0201	0.0
adj_em_5	1.5098	4.5258	2.9352	6.9785	0.0

```
Model 2: AdjEM + win percentage
```

	Coefficient (log-odds)	Odds ratio	OR 95% CI lower (naive)	OR 95% CI upper (naive)
Intercept	-4.6502	0.0096	0.0046	0.0201
adj_em_5	1.4666	4.3346	2.3214	8.0937
win_pct_10pp	0.0736	1.0763	0.4954	2.3385

```
Model 3: AdjO + AdjD
```

	Coefficient (log-odds)	Odds ratio	OR 95% CI lower (naive)	OR 95% CI upper (naive)	P-valu
Intercept	-7.6104	0.0005	0.0001	0.0023	0.0
adj_o_5	1.5484	4.7037	2.8783	7.6868	0.0
adj_d_5	-1.4358	0.2379	0.1291	0.4386	0.0

```
Model 1, +5 AdjEM: odds multiplied by 4.53
Model 2, +5 AdjEM (holding win% fixed): odds multiplied by 4.33
Model 2, +10 percentage points win percentage (holding AdjEM fixed): odds multiplied by 1.08
Model 3, +5 AdjO (holding AdjD fixed): odds multiplied by 4.70
Model 3, +5 AdjD (holding AdjO fixed): odds multiplied by 0.24
```

How to read these estimates: If Model 1's AdjEM odds ratio exceeds 1, teams with higher AdjEM have higher estimated championship odds. If Model 2's win-percentage odds ratio is close to 1, adding 10 percentage points to win percentage has little estimated association with winning *after accounting for AdjEM*. A negative coefficient for AdjD is consistent with stronger defenses (lower points allowed per 100 possessions) being associated with better championship prospects.

Comparison of initial models

AIC compares in-sample fit while penalizing extra parameters; lower is better. Since Model 2 adds one variable to Model 1, we also use a likelihood-ratio test to ask whether the additional term meaningfully improves fit. These measures describe **in-sample** performance and are not proof that a model predicts new seasons well.

```
comparison = pd.DataFrame({
    "Model": ["1: AdjEM", "2: AdjEM + win%", "3: AdjO + AdjD"],
    "Number of coefficients": [len(m.params) for m in (m1, m2, m3)],
    "Log-likelihood": [m.llf for m in (m1, m2, m3)],
    "AIC (lower is better)": [m.aic for m in (m1, m2, m3)],
    "Converged": [m.converged for m in (m1, m2, m3)],
})
display(comparison.round(3))

lr_stat = 2 * (m2.llf - m1.llf)
lr_pvalue = chi2.sf(lr_stat, df=1)
print(f"Likelihood-ratio test, Model 2 vs Model 1: "
      f"statistic={lr_stat:.3f}, p={lr_pvalue:.4f}")
```

	Model	Number of coefficients	Log-likelihood	AIC (lower is better)	Converged
0	1: AdjEM	2	-72.941	149.882	True
1	2: AdjEM + win%	3	-72.924	151.848	True
2	3: AdjO + AdjD	3	-72.886	151.772	True

```
Likelihood-ratio test, Model 2 vs Model 1: statistic=0.034, p=0.8527
```

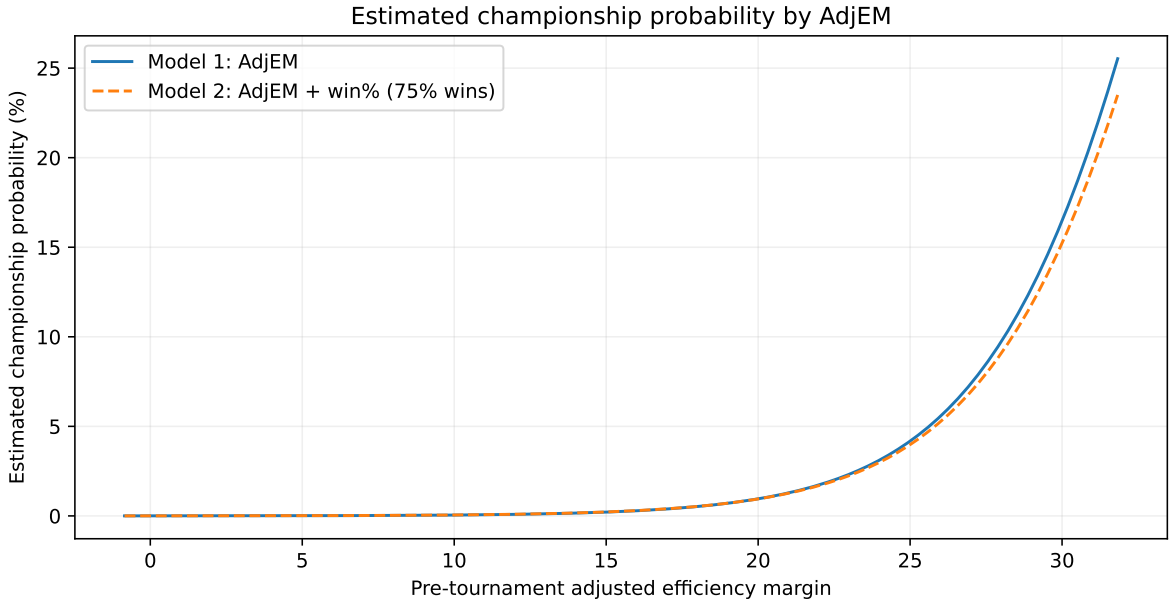
What do these estimates mean as probabilities?

A change in odds is not the same as a change in probability. To make the results easier to interpret, we use Model 1 to estimate championship probabilities for a range of AdjEM values, and Model 2 to estimate probabilities while holding win percentage at 75%.

```
adj_values = np.array([15, 20, 25, 30, 35], dtype=float)
scenarios = pd.DataFrame({
    "AdjEM": adj_values,
    "adj_em_5": (adj_values - 20) / 5,
    "win_pct_10pp": np.zeros(len(adj_values)),  # win percentage = 75%
})
scenarios["Model 1 probability (%)"] = 100 * m1.predict(scenarios)
scenarios["Model 2 probability at 75% wins (%)"] = 100 * m2.predict(scenarios)
display(scenarios[["AdjEM", "Model 1 probability (%)",
                   "Model 2 probability at 75% wins (%)"]].round(2))

x = np.linspace(data["adj_em"].quantile(0.05),
                data["adj_em"].quantile(0.99), 200)
plot_data = pd.DataFrame({
    "adj_em_5": (x - 20) / 5,
    "win_pct_10pp": np.zeros(len(x)),
})
fig, ax = plt.subplots(figsize=(8.5, 4.5))
ax.plot(x, 100 * m1.predict(plot_data), label="Model 1: AdjEM")
ax.plot(x, 100 * m2.predict(plot_data), linestyle="--",
        label="Model 2: AdjEM + win% (75% wins)")
ax.set(xlabel="Pre-tournament adjusted efficiency margin",
       ylabel="Estimated championship probability (%)",
       title="Estimated championship probability by AdjEM")
ax.grid(alpha=0.2)
ax.legend()
plt.tight_layout()
plt.show()
```

	AdjEM	Model 1 probability (%)	Model 2 probability at 75% wins (%)
0	15.0	0.21	0.22
1	20.0	0.95	0.95
2	25.0	4.18	3.98
3	30.0	16.47	15.23
4	35.0	47.16	43.77



These are estimated probabilities from a standard team-level logistic regression. **They are not yet fully calibrated tournament-winning probabilities:** the model fits teams independently, so the predicted probabilities across a given year's tournament need not add up to 100%, even though there can be only one champion.

Diagnostics and limitations of the initial model

Milestone 14 examines the logistic regression assumptions in more detail. Here we briefly check whether the **models actually fitted for Milestone 15** show obvious identifiability or convergence problems.

```
# VIF is calculated for the actual two-predictor designs, not all CSV columns.
def vif_table(cols):
    X = sm.add_constant(data[cols], has_constant="add")
    return pd.DataFrame({
        "Variable": cols,
        "VIF": [variance_inflation_factor(X.to_numpy(), i)
                for i in range(1, X.shape[1])],
    })

print("VIF - Model 2")
display(vif_table(["adj_em_5", "win_pct_10pp"]).round(2))
print("VIF - Model 3")
```

```
display(vif_table(["adj_o_5", "adj_d_5"]).round(2))

print("Convergence:", {"Model 1": m1.converged,
                       "Model 2": m2.converged,
                       "Model 3": m3.converged})
print("Fitted probabilities within 0-1:",
      all(np.asarray(m.predict(data)).min() >= 0 and
          np.asarray(m.predict(data)).max() <= 1 for m in (m1, m2, m3)))

# Influence is examined for the primary model; high influence isn't necessarily bad data.
influence = m1.get_influence(observed=True)
cooks_d = np.asarray(influence.cooks_distance[0])
flag_threshold = 4 / len(data)
print(f"Model 1 observations with Cook's D > 4/n ({flag_threshold:.5f}): "
      f"{int((cooks_d > flag_threshold).sum())}")
display(data.assign(cooks_d=cooks_d)
        .sort_values("cooks_d", ascending=False)
        [["season", "team", "champion", "adj_em", "cooks_d"]].head(5))
```

VIF - Model 2

	Variable	VIF
0	adj_em_5	1.48
1	win_pct_10pp	1.48

VIF - Model 3

	Variable	VIF
0	adj_o_5	1.01
1	adj_d_5	1.01

```
Convergence: {'Model 1': True, 'Model 2': True, 'Model 3': True}
Fitted probabilities within 0-1: True
Model 1 observations with Cook's D > 4/n (0.00298): 32
```

	season	team	champion	adj_em	cooks_d
3129	2014	Connecticut	1	17.8037	0.102863
2097	2011	Connecticut	1	20.1352	0.067850
414	2006	Florida	1	21.6322	0.050931
6656	2025	Duke	0	36.3204	0.044600
3552	2015	Kentucky	0	36.2114	0.042540

Our most important cautions are: **(1)** only 21 of the teams are champions; **(2)** exactly one team wins per year, so outcomes within a season are dependent; **(3)** some tournament entrants are absent from the exported field; and **(4)** the historical feature snapshots still need an independent provenance check for leakage. Model-based p-values and confidence intervals should be interpreted with those limitations in mind. Any influential champions should be inspected, not automatically deleted.

Preliminary test on later seasons

As a first check of generalization, we fit the AdjEM model on 2005–2021 (excluding 2020) and test it on **2022–2026**. We do **not** randomly split team-seasons, since that could put teams from the same championship tournament in both the training and test sets.

We compare the model's Brier score (mean squared error of estimated probabilities; lower is better) against a simple reference predicting `1 / recorded tournament teams` for each test year. We also report where the actual champion ranked based on the model's scores. This is only a five-season holdout, not a definitive evaluation of MacFax.

```
train = data.loc[data["season"] <= 2021].copy()
test = data.loc[data["season"] >= 2022].copy()

if train["champion"].sum() < 5 or test["champion"].sum() < 1:
    raise ValueError("Not enough positive examples for the selected holdout.")

heldout_fit = smf.glm("champion ~ adj_em_5", data=train,
                      family=sm.families.Binomial()).fit()
test["estimated_p"] = heldout_fit.predict(test)
test["uniform_p"] = 1 / test.groupby("season")["team_id"].transform("count")

brier_model = np.mean((test["champion"] - test["estimated_p"]) ** 2)
brier_uniform = np.mean((test["champion"] - test["uniform_p"]) ** 2)

# One champion per year: a higher rank means the champion was less highly rated.
```

```python
test["predicted_rank"] = (
    test.groupby("season")["estimated_p"]
        .rank(method="min", ascending=False)
        .astype(int)
)
champion_ranks = test.loc[test["champion"] == 1,
                          ["season", "team", "predicted_rank", "estimated_p"]]

print(f"Training champions: {int(train['champion'].sum())} "
      f"({train['season'].nunique()} seasons)")
print(f"Test champions: {int(test['champion'].sum())} "
      f"({test['season'].nunique()} seasons)")
print(f"Holdout Brier score - AdjEM model: {brier_model:.5f}")
print(f"Holdout Brier score - uniform field baseline: {brier_uniform:.5f}")
print(f"Champions ranked in top 5: "
      f"{int((champion_ranks['predicted_rank'] <= 5).sum())} "
      f"of {len(champion_ranks)}")
display(champion_ranks.sort_values("season").round(4))
```

```
Training champions: 16 (16 seasons)
Test champions: 5 (5 seasons)
Holdout Brier score - AdjEM model: 0.01550
Holdout Brier score - uniform field baseline: 0.01548
Champions ranked in top 5: 5 of 5
```

	season	team	predicted_rank	estimated_p
5640	2022	Kansas	5	0.0412
5923	2023	Connecticut	5	0.0262
6282	2024	Connecticut	2	0.2148
6670	2025	Florida	2	0.4386
7118	2026	Michigan	3	0.5001

The Brier scores and rankings answer **different** questions. A model can rank the eventual champion highly without assigning accurate absolute championship probabilities. Also, because we have only five test tournaments, any performance differences are unstable. The current analysis does **not** establish that the MacFax checklist itself has been validated.

What story is starting to take shape?

The fitted results suggest that **pre-tournament adjusted efficiency margin is strongly associated with championship outcomes**. Teams that are substantially more efficient than their opponents appear more likely to win the title. Our second model tests whether raw win percentage improves that explanation; its incremental value can be compared directly using the coefficient estimate and AIC. Our third model examines whether the balance of offensive and defensive efficiency offers a different explanation of championship-level teams.

The most meaningful early result is not that we have identified a guaranteed champion. It's that **overall efficiency provides a useful baseline for separating serious contenders from the rest of the tournament field**, while other measures need to demonstrate additional value rather than merely correlate with team strength.

The paragraph below uses the *actual fitted coefficients* so the interpretation stays accurate if the shared dataset changes.

```
or_adj_1 = float(np.exp(m1.params["adj_em_5"]))
or_adj_2 = float(np.exp(m2.params["adj_em_5"]))
or_win_2 = float(np.exp(m2.params["win_pct_10pp"]))

message = f"""
**Results from our fitted models.** In Model 1, a five-point increase in AdjEM
is associated with **{or_adj_1:.2f} times the championship odds**. In Model 2,
the corresponding adjusted-efficiency odds ratio is **{or_adj_2:.2f}**, and a
10-percentage-point increase in win percentage, holding AdjEM fixed, has an
estimated odds ratio of **{or_win_2:.2f}**. The models' AIC values are **{m1.aic:.2f}**
(AdjEM), **{m2.aic:.2f}** (AdjEM + win percentage), and **{m3.aic:.2f}**
(offense + defense). Adding win percentage produces a likelihood-ratio
p-value of **{lr_pvalue:.3f}**; this is exploratory given our sampling structure.

**Out-of-sample caution.** On the five held-out tournaments, the AdjEM-only
model had a Brier score of **{brier_model:.5f}**, compared with **{brier_uniform:.5f}**
for the uniform-field baseline, even though it ranked the champion in the
top five in **{int((champion_ranks['predicted_rank'] <= 5).sum())} of
{len(champion_ranks)}** test seasons. This suggests contender ranking may
be more promising than the current model's uncalibrated absolute probabilities.
"""
display(Markdown(message))
```

Results from our fitted models. In Model 1, a five-point increase in AdjEM is associated with **4.53 times the championship odds**. In Model 2, the corresponding adjusted-efficiency odds ratio is **4.33**, and a 10-percentage-point increase in win percentage, holding AdjEM fixed,

has an estimated odds ratio of **1.08**. The models' AIC values are **149.88** (AdjEM), **151.85** (AdjEM + win percentage), and **151.77** (offense + defense). Adding win percentage produces a likelihood-ratio p-value of **0.853**; this is exploratory given our sampling structure.

Out-of-sample caution. On the five held-out tournaments, the AdjEM-only model had a Brier score of **0.01550**, compared with **0.01548** for the uniform-field baseline, even though it ranked the champion in the top five in **5 of 5** test seasons. This suggests contender ranking may be more promising than the current model's uncalibrated absolute probabilities.

Conclusion and next steps

Our initial logistic regression supports the idea that **adjusted efficiency is an informative summary of championship-caliber teams**, but the analysis does not prove causation or produce a trustworthy finished championship probability system. The win-percentage and separate offense/defense models help us ask whether additional statistics contribute information that overall efficiency misses. Because only 21 championship outcomes are available, we should avoid adding many highly correlated checklist variables to one unregularized model.

For the broader MacFax Crystal Ball project, our next steps are to verify pre-tournament data provenance, correct missing historical tournament-team coverage, evaluate the existing 15 checklist rules in genuinely held-out seasons, and investigate probability calibration or bracket-based simulation. **We should not remove checklist criteria solely because they correlate with each other:** the regression and the checklist perform different jobs, and changes should be based on independent predictive performance.

Reproducibility: Save this file at the root of `data_5600project2`, next to the `data/` folder. With the shared CSV present, render using:

```
quarto render milestone15_champion_checklist.qmd --to html
```

The Python environment needs `numpy`, `pandas`, `scipy`, `statsmodels`, `matplotlib`, `ipython`, `ipykernel`, and `jupyter`. This report does not require MacFax's PostgreSQL database or the private database backup.